Portable Pantry

Class List for Section 4 Class Diagram (Team A), revised

This list replaces the earlier class list and class diagram. It follows the Deliverable 2 feedback: no RegisteredUser, GuestUser, or CompletedRecipe classes and no registered flag; domain classes hold data only; getters and setters are assumed, so values are passed through constructors; every method has typed parameters and a return type; and the design has controllers and persistence. Recipes come from Ollama (generation) and TheMealDB (search by ingredients). Supabase stores data and handles authentication.

Notation. - private, + public. *Type?* means the value may be empty (None). Every domain class has a constructor that takes all of its attributes in the order listed, and read-only getters for each. Guests receive a temporary User, UserProfile, and Pantry that are never saved.

Layering. Controller → Service → Repository → DatabaseManager → Supabase. Controllers never call repositories. Every dependency is passed in through the constructor.

Total: 43 classes (11 domain, 8 filter, 4 recipe source, 3 factory, 7 service, 5 persistence, 5 controller).

1. Domain Classes (11)

Class	Responsibility	Attributes	Methods
User	A person using the site, signed in or guest	-userId: String -username: String -email: String -profile: UserProfile	*none*
UserProfile	A user's display name and pantry	-displayName: String -pantry: Pantry	*none*
Pantry	Everything a user keeps: ingredients, favorites, allergies, restrictions, saved and completed recipes	-savedIngredients: List<Ingredient> -favoriteIngredients: List<Ingredient> -allergies: List<Allergy> -dietaryRestrictions: List<DietaryRestriction> -savedRecipes: List<Recipe> -completedRecipes: List<Recipe>	*none*
Ingredient	A single food item, with an amount when used in a recipe	-ingredientId: String -name: String -category: String -quantity: float -unit: String	*none*
Allergy	A selectable allergy	-allergyId: String -name: String	*none*
DietaryRestriction	A selectable dietary restriction	-restrictionId: String -name: String	*none*
Recipe	A generated or searched recipe	-recipeId: String -title: String -description: String -category: String -imageUrl: String? -source: String ("ollama" or "themealdb") -servings: int -calories: int? (per serving) -difficulty: int? (1-4) -spiceLevel: int? (1-5) -ingredients: List<Ingredient> -steps: List<RecipeStep>	*none*
RecipeStep	One preparation instruction	-stepNumber: int -instructionText: String	*none*
RecipeRating	A user's star rating of a recipe	-user: User -recipe: Recipe -stars: int (1-5)	*none*

Class	Responsibility	Attributes	Methods
PersonalNote	A user's private note on a recipe	-user: User -recipe: Recipe -noteText: String -updatedAt: DateTime	*none*
RecipeQuery	Everything needed for one generate request	-ingredients: List<Ingredient> -filters: List<Filter> -portionSize: int	*none*

2. Filter Classes (8)

Filter is abstract. Each subclass receives its values through its constructor and overrides apply(). TheMealDB has no calorie, difficulty, or spice data, so CalorieFilter, DifficultyFilter, and SpiceLevelFilter only affect Ollama recipes; recipes with no value are kept.

Class	Responsibility	Attributes	Methods
Filter **«abstract»**	Defines the shared filter operation	*none*	+apply(recipes: List<Recipe>): List<Recipe> «abstract»
AllergyFilter	Removes recipes containing the user's allergies	-allergies: List<Allergy>	+apply(recipes: List<Recipe>): List<Recipe>
DietaryRestrictionFilter	Removes recipes that break the user's restrictions	-restrictions: List<DietaryRestriction>	+apply(recipes: List<Recipe>): List<Recipe>
FoodCategoryFilter	Keeps recipes in the chosen categories	-categories: List<String>	+apply(recipes: List<Recipe>): List<Recipe>
CalorieFilter	Keeps recipes inside a calorie range	-minCalories: int -maxCalories: int	+apply(recipes: List<Recipe>): List<Recipe>
DifficultyFilter	Keeps recipes at or below a difficulty	-maxDifficulty: int (1-4)	+apply(recipes: List<Recipe>): List<Recipe>
SpiceLevelFilter	Keeps recipes at or below a spice level	-maxSpiceLevel: int (1-5)	+apply(recipes: List<Recipe>): List<Recipe>
PantryIngredientFilter	Keeps recipes missing at most maxMissing ingredients, fewest missing first	-available: List<Ingredient> -maxMissing: int	+apply(recipes: List<Recipe>): List<Recipe> -countMissing(recipe: Recipe): int

3. Recipe Source Classes (4)

Each interface lets a new source (for example AllRecipes or BBC) be added without changing the services. The concrete classes convert external data into Recipe objects (Adapter pattern).

Class	Responsibility	Attributes	Methods
RecipeGenerator **«interface»**	Creates new recipes from a query	*none*	+generateRecipes(query: RecipeQuery): List<Recipe>
OllamaRecipeGenerator	Generates recipes with a local Ollama model	-baseUrl: String -model: String -recipeCount: int	+generateRecipes(query: RecipeQuery): List<Recipe> -buildPrompt(query: RecipeQuery): String -requestRecipes(prompt: String): String -parseRecipes(response: String): List<Recipe>
RecipeSearcher **«interface»**	Finds existing recipes by ingredients	*none*	+searchByIngredients(ingredients: List<Ingredient>): List<Recipe>
MealDbRecipeSearcher	Searches TheMealDB: one filter call per ingredient, ranks by matches, then looks up full details for the best candidates	-baseUrl: String -apiKey: String -maxLookups: int	+searchByIngredients(ingredients: List<Ingredient>): List<Recipe> -filterByIngredient(name: String): List<String> -lookupRecipe(id: String): Recipe -toRecipe(json: Map<String, Object>): Recipe

4. Factory Classes (3)

Class	Responsibility	Attributes	Methods
RecipeGeneratorFactory	Creates the RecipeGenerator for a source name	-settings: Map<String, String>	+createGenerator(type: String): RecipeGenerator
APIFactory	Creates the RecipeSearcher for an API name	-settings: Map<String, String>	+createSearcher(type: String): RecipeSearcher
FilterFactory	Builds filter objects from the user's filter settings and pantry	*none*	+createFilters(settings: Map<String, Object>, pantry: Pantry): List<Filter>

5. Service Classes (7)

Services hold all application logic. Because domain classes have no setters, a service that changes a pantry builds a new Pantry and saves it through the repository.

Class	Responsibility	Attributes	Methods
RecommendationService	Generates recipes, applies the query's filters, saves them, and regenerates with the same criteria	-generatorFactory: RecipeGeneratorFactory -recipeService: RecipeService -recipeRepository: RecipeRepository	+generateRecipes(query: RecipeQuery): List<Recipe> +regenerateRecipes(query: RecipeQuery, shown: List<Recipe>): List<Recipe>
RecipeService	Searches, filters, finds, and scales recipes	-apiFactory: APIFactory -filterFactory: FilterFactory -recipeRepository: RecipeRepository	+buildQuery(user: User, ingredients: List<Ingredient>, settings: Map<String, Object>, portionSize: int): RecipeQuery +searchRecipes(ingredients: List<Ingredient>): List<Recipe> +filterRecipes(recipes: List<Recipe>, filters: List<Filter>): List<Recipe> +getRecipe(recipeId: String): Recipe +scaleRecipe(recipe: Recipe, servings: int): Recipe
UserRecipeService	A user's saved, completed, rated, and noted recipes	-userRecipeRepository: UserRecipeRepository	+saveRecipe(user: User, recipe: Recipe): bool +removeSavedRecipe(user: User, recipe: Recipe): bool +rateRecipe(user: User, recipe: Recipe, stars: int): bool +addNote(user: User, recipe: Recipe, noteText: String): bool +markCompleted(user: User, recipe: Recipe): bool +shareRecipe(recipe: Recipe, email: String): bool +getSavedRecipes(user: User): List<Recipe> +getCompletedRecipes(user: User): List<Recipe> +getRating(user: User, recipe: Recipe): RecipeRating? +getNote(user: User, recipe: Recipe): PersonalNote?
IngredientService	Manages the ingredients and favorites in a user's pantry	-pantryRepository: PantryRepository	+addIngredient(user: User, ingredient: Ingredient): bool +removeIngredient(user: User, ingredientId: String): bool +editIngredient(user: User, ingredient: Ingredient): bool +getIngredients(user: User): List<Ingredient> +markFavorite(user: User, ingredientId: String): bool +unmarkFavorite(user: User, ingredientId: String): bool
ProfileService	Manages the display name, allergies, and dietary restrictions	-userRepository: UserRepository -pantryRepository: PantryRepository	+getProfile(user: User): UserProfile +updateDisplayName(user: User, displayName: String): bool +updateAllergies(user: User, allergies: List<Allergy>): bool +updateDietaryRestrictions(user: User, restrictions: List<DietaryRestriction>): bool

Class	Responsibility	Attributes	Methods
AuthService	Sign-up, sign-in, sessions, and passwords through Supabase Auth	-databaseManager: DatabaseManager -userRepository: UserRepository	+createAccount(email: String, username: String, password: String): User +login(email: String, password: String): User +logout(user: User): bool +createGuest(): User +getSessionUser(sessionToken: String): User? +changePassword(user: User, oldPassword: String, newPassword: String): bool +resetPassword(email: String): bool
AccountService	Edits, resets, and deletes accounts	-databaseManager: DatabaseManager -userRepository: UserRepository -pantryRepository: PantryRepository -userRecipeRepository: UserRecipeRepository	+editAccount(user: User, username: String, email: String): User +resetAccount(user: User): bool +deleteAccount(user: User): bool

6. Persistence Classes (5)

Each repository reads and writes one area of Supabase data and turns table rows into domain objects (Repository pattern). All four share one DatabaseManager connection.

Class	Responsibility	Attributes	Methods
DatabaseManager	Creates the single Supabase connection from the .env settings	-url: String -key: String -client: Client	+getClient(): Client
UserRepository	Users and their profiles	-db: DatabaseManager	+findById(userId: String): User? +findByEmail(email: String): User? +save(user: User): void +delete(userId: String): void
PantryRepository	Each user's pantry	-db: DatabaseManager	+findByUserId(userId: String): Pantry +save(userId: String, pantry: Pantry): void +deleteByUserId(userId: String): void
RecipeRepository	Stored recipes (generated recipes are saved so they can be opened later)	-db: DatabaseManager	+findById(recipeId: String): Recipe? +save(recipe: Recipe): void +saveAll(recipes: List<Recipe>): void
UserRecipeRepository	The user-to-recipe link table: saved, completed, ratings, and notes	-db: DatabaseManager	+addSaved(userId: String, recipeId: String): void +removeSaved(userId: String, recipeId: String): void +findSaved(userId: String): List<Recipe> +addCompleted(userId: String, recipeId: String): void +findCompleted(userId: String): List<Recipe> +saveRating(rating: RecipeRating): void +findRating(userId: String, recipeId: String): RecipeRating? +saveNote(note: PersonalNote): void +findNote(userId: String, recipeId: String): PersonalNote? +deleteAllForUser(userId: String): void

7. Controller Classes (5)

Controllers are Flask blueprints. Each handler reads the request, gets the signed-in User from AuthService, calls a service, and returns a JSON Response. They never call repositories.

Class	Responsibility	Attributes	Methods
AuthController	Routes under /api/auth	-authService: AuthService	+signup(): Response +login(): Response +logout(): Response +continueAsGuest(): Response +changePassword(): Response +resetPassword(): Response

Class	Responsibility	Attributes	Methods
AccountController	Routes under /api/account	-authService: AuthService -accountService: AccountService -profileService: ProfileService	+editAccount(): Response +resetAccount(): Response +deleteAccount(): Response +getProfile(): Response +updateProfile(): Response
RecipeController	Routes under /api/recipes	-authService: AuthService -recommendationService: RecommendationService -recipeService: RecipeService	+generate(): Response +regenerate(): Response +search(): Response +getRecipe(recipeId: String): Response +scaleRecipe(recipeId: String): Response
UserRecipeController	Routes under /api/user-recipes	-authService: AuthService -userRecipeService: UserRecipeService	+saveRecipe(recipeId: String): Response +removeSavedRecipe(recipeId: String): Response +rateRecipe(recipeId: String): Response +addNote(recipeId: String): Response +markCompleted(recipeId: String): Response +shareRecipe(recipeId: String): Response +getSavedRecipes(): Response +getCompletedRecipes(): Response
PantryController	Routes under /api/pantry	-authService: AuthService -ingredientService: IngredientService	+getIngredients(): Response +addIngredient(): Response +editIngredient(ingredientId: String): Response +removeIngredient(ingredientId: String): Response +markFavorite(ingredientId: String): Response +unmarkFavorite(ingredientId: String): Response

8. Main Associations

- User has one UserProfile (composition, 1 to 1). UserProfile has one Pantry (composition, 1 to 1).
- Pantry aggregates 0..* Ingredients, Allergies, DietaryRestrictions, and Recipes (saved and completed).
- Recipe is composed of 1..* RecipeSteps and aggregates 1..* Ingredients.
- RecipeRating and PersonalNote each associate one User with one Recipe.
- RecipeQuery aggregates 1..* Ingredients and 0..* Filters.
- The seven filters inherit from Filter. OllamaRecipeGenerator realizes RecipeGenerator, and MealDbRecipeSearcher realizes RecipeSearcher.
- RecipeGeneratorFactory «creates» RecipeGenerator and OllamaRecipeGenerator. APIFactory «creates» RecipeSearcher and MealDbRecipeSearcher. FilterFactory «creates» Filter and its subclasses.
- RecommendationService associates with RecipeGeneratorFactory, RecipeService, and RecipeRepository. RecipeService associates with APIFactory, FilterFactory, and RecipeRepository.
- UserRecipeService uses UserRecipeRepository. IngredientService uses PantryRepository. ProfileService, AuthService, and AccountService use UserRepository (AccountService also uses PantryRepository and UserRecipeRepository). AuthService associates with User, not UserProfile.
- Each repository and AuthService depend on DatabaseManager, which depends on Supabase.
- Each controller associates with AuthService and the service(s) listed in its attributes.

9. Design Patterns

- **Factory:** RecipeGeneratorFactory, APIFactory, and FilterFactory create objects from a type name or settings, so services never name a concrete class.
- **Adapter:** OllamaRecipeGenerator and MealDbRecipeSearcher convert each external API's format into Recipe.
- **Repository:** four repositories keep all Supabase access out of the services.
- **Dependency injection:** every class receives its collaborators through its constructor.

10. Open Decisions

- **passwordHash on User.** Left out because Supabase Auth stores passwords. Add it back only if the team decides to manage passwords itself.
- **Filter hierarchy.** Keep the Filter subclasses, or flatten them into one FilterService. This list keeps the subclasses.
- **Supabase tables.** Table design, including the user-to-recipe link table behind UserRecipeRepository, is still to be done.